MATERIA

DESARROLLO DE VIDEOJUEGOS (INF761)

UNIDAD 2

Mecánicas de Juego y Sistemas de Gameplay

Material de Estudio Teórico

CONTENIDO DE LA UNIDAD

2.1 Diseño de mecánicas
Game loops, estado de juego, condiciones y feedback

2.2 Patrones de diseño para juegos
Singleton, Observer, State Machine, Command, Object Pool

2.3 Sistemas de inventario y economía
Estructuras de datos, JSON/PlayerPrefs, serialización

2.4 Interfaces de usuario (UI Toolkit / uGUI)
Menús, HUD, barras de vida, diálogos, localización

2.5 Gestión de escenas y flujo de juego
Carga aditiva, pantallas de carga, transiciones, persistencia

2.6 Prototipo 1: Juego 2D Top-Down
Integración de mecánicas, UI y gestión de escenas

COMPETENCIA DE LA UNIDAD

Implementar mecánicas de juego robustas utilizando patrones de diseño de software, sistemas de interfaz de usuario y gestión de escenas para crear experiencias de gameplay completas.

2. Mecánicas de Juego y Sistemas de Gameplay

2.1 Diseño de mecánicas: game loops, estado de juego, condiciones y feedback

El diseño de mecánicas es la disciplina que define cómo interactúa el jugador con el sistema del juego. Una mecánica bien diseñada es simple de entender, difícil de dominar y genera decisiones significativas. Las mecánicas no existen de forma aislada: se interconectan formando el sistema de gameplay completo.

El Game Loop

El game loop es el ciclo fundamental de ejecución de cualquier videojuego. Define el flujo continuo: el jugador actúa, el sistema responde, el mundo cambia, el jugador vuelve a actuar. Este ciclo puede analizarse en dos niveles:

- **Micro loop (loop de frame):** Update → procesar input → actualizar lógica → renderizar → esperar siguiente frame. Unity gestiona este loop internamente a través del ciclo MonoBehaviour.
- **Macro loop (loop de sesión):** El jugador inicia el juego → explora y completa objetivos → alcanza victoria o derrota → vuelve al inicio o progresa al siguiente nivel.

> **DEFINICIÓN — CORE LOOP**
> El core loop es la secuencia de acciones fundamentales que el jugador repite durante toda la sesión. En un RPG: explorar → combatir → obtener recompensas → mejorar personaje → explorar. En un puzzle: analizar → intentar solución → fallar o resolver → siguiente nivel. Un core loop sólido es adictivo y satisfactorio por sí solo, independientemente del contexto narrativo.

Estado de juego (Game State)

El estado de juego es el conjunto completo de variables que describen la situación actual del juego en un momento dado. Gestionar el estado correctamente es esencial para implementar guardado, multijugador, replay y depuración.

Categoría de estado	Ejemplos de variables
Estado del jugador	Posición, HP, mana, inventario, experiencia, nivel
Estado del mundo	Enemigos activos, puertas abiertas, objetos recogidos, tiempo
Estado de la partida	Puntuación, vidas restantes, timer, oleada actual
Estado de la UI	Menú activo, panel visible, elemento seleccionado
Estado de progresión	Misiones completadas, logros, habilidades desbloqueadas

Condiciones de victoria y derrota

Las condiciones de victoria y derrota son los criterios que terminan la partida. Deben ser claras, justas y comunicadas al jugador con anticipación para no generar frustración.

- **Victoria explícita:** Eliminar al jefe final, llegar al punto de extracción, acumular X puntos antes del límite de tiempo.
- **Victoria implícita:** Explorar todo el mapa, encontrar todos los secretos, completar todos los logros opcionales.

- **Derrota por agotamiento de recurso:** HP = 0, tiempo agotado, base destruida, combustible terminado.
- **Derrota por fallo de objetivo:** El rehén murió, la bomba explotó, el objetivo escapó del área.

Feedback al jugador

El feedback es la respuesta inmediata y perceptible que el sistema da al jugador ante sus acciones. Un feedback efectivo cierra el loop de comunicación entre jugador y sistema, haciendo que el juego se sienta vivo y responsivo.

Tipo de feedback	Canal	Ejemplo en un shooter 2D
Visual	Pantalla	Destello rojo en bordes al recibir daño
Sonoro	Audio	Sonido de impacto, música tensa al HP bajo
Háptico	Gamepad	Vibración al disparar o recibir golpe
Numérico	UI	Número de daño flotante sobre el enemigo
Narrativo	Diálogo/HUD	Personaje exclama al quedar con poca vida
Animación	Personaje	Hit stun: retroceso y flash blanco al impactar

2.2 Patrones de diseño para juegos: Singleton, Observer, State Machine, Command y Object Pooling

Los patrones de diseño son soluciones probadas a problemas recurrentes de software. En desarrollo de videojuegos, ciertos patrones son especialmente relevantes por las restricciones de rendimiento en tiempo real y la complejidad de los sistemas de gameplay interconectados.

Singleton

El Singleton garantiza que una clase tenga una única instancia accesible globalmente. En Unity se usa para managers (GameManager, AudioManager, UIManager) que deben existir durante toda la sesión de juego y ser accesibles desde cualquier script.

```
public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }
    public int Score  { get; private set; }
    public int Lives  { get; private set; } = 3;

    void Awake()
    {
        if (Instance != null && Instance != this) { Destroy(gameObject); return;
}
        Instance = this;
        DontDestroyOnLoad(gameObject);
    }

    public void AddScore(int points)
    {
        Score += points;
        UIManager.Instance.UpdateScoreDisplay(Score);
    }

    public void LoseLife()
    {
```

```
        Lives--;
        if (Lives <= 0) SceneManager.LoadScene("GameOver");
    }
}
```

IMPORTANTE
El Singleton es conveniente pero introduce acoplamiento global. Úsalo solo para managers verdaderamente globales. Evita convertir todas las clases en Singletons: dificulta el testing unitario y crea dependencias ocultas difíciles de rastrear.

Observer (Patrón Observador)

El Observer desacopla emisores de eventos de sus receptores. El emisor no necesita conocer quién está escuchando. En Unity se implementa con C# events/Actions o con ScriptableObjects como canales de eventos reutilizables.

```
// Canal de eventos como ScriptableObject (Event Channel Pattern)
[CreateAssetMenu(menuName = "Events/Game Event")]
public class GameEvent : ScriptableObject
{
    private List<GameEventListener> listeners = new();
    public void Raise()
    {
        for (int i = listeners.Count - 1; i >= 0; i--)
            listeners[i].OnEventRaised();
    }
    public void RegisterListener(GameEventListener l)   => listeners.Add(l);
    public void UnregisterListener(GameEventListener l) => listeners.Remove(l);
}

public class GameEventListener : MonoBehaviour
{
    [SerializeField] private GameEvent   gameEvent;
    [SerializeField] private UnityEvent  response;
    void OnEnable()  => gameEvent.RegisterListener(this);
    void OnDisable() => gameEvent.UnregisterListener(this);
    public void OnEventRaised() => response.Invoke();
}
```

State Machine (Máquina de Estados)

La State Machine modela comportamientos que cambian según el estado actual del objeto. Es ideal para IA de enemigos, comportamiento del jugador (caminar/saltar/atacar) y flujo de pantallas de UI.

```
public enum EnemyState { Idle, Patrol, Chase, Attack, Dead }

public class EnemyAI : MonoBehaviour
{
    private EnemyState state = EnemyState.Idle;
    private Transform player;
    [SerializeField] private float detectionRange = 8f;
    [SerializeField] private float attackRange    = 1.5f;

    void Update() => HandleState();

    void HandleState()
    {
```

```
        float dist = Vector2.Distance(transform.position, player.position);
        switch (state)
        {
            case EnemyState.Idle:
                if (dist < detectionRange) ChangeState(EnemyState.Chase);
                break;
            case EnemyState.Chase:
                MoveTowardsPlayer();
                if (dist < attackRange)         ChangeState(EnemyState.Attack);
                if (dist > detectionRange * 1.5f) ChangeState(EnemyState.Patrol);
                break;
            case EnemyState.Attack:
                AttackPlayer();
                if (dist > attackRange) ChangeState(EnemyState.Chase);
                break;
        }
    }

    void ChangeState(EnemyState newState) { state = newState; }
    void MoveTowardsPlayer() { /* lógica de movimiento */ }
    void AttackPlayer()      { /* lógica de ataque     */ }
}
```

Command (Patrón Comando)

El Command encapsula una acción como objeto, permitiendo implementar historial de acciones (undo/redo), acciones diferidas, macros y replay. Especialmente útil en juegos de estrategia y puzzles.

```
public interface ICommand { void Execute(); void Undo(); }

public class MoveCommand : ICommand
{
    private Transform target;
    private Vector3 prevPos, newPos;
    public MoveCommand(Transform t, Vector3 dest) { target = t; prevPos =
t.position; newPos = dest; }
    public void Execute() => target.position = newPos;
    public void Undo()    => target.position = prevPos;
}

public class CommandManager : MonoBehaviour
{
    private Stack<ICommand> history = new();
    public void ExecuteCommand(ICommand cmd) { cmd.Execute(); history.Push(cmd);
}
    public void UndoLast() { if (history.Count > 0) history.Pop().Undo(); }
}
```

Object Pooling

El Object Pool pre-instancia objetos al inicio del juego y los recicla en lugar de crearlos y destruirlos continuamente. Elimina la presión sobre el Garbage Collector (GC), evitando los stutters causados por la recolección de basura durante el gameplay.

```
public class ObjectPool : MonoBehaviour
{
    [SerializeField] private GameObject prefab;
    [SerializeField] private int        initialSize = 20;
    private Queue<GameObject> pool = new();
```

```
    void Awake()
    {
        for (int i = 0; i < initialSize; i++)
        {
            var obj = Instantiate(prefab);
            obj.SetActive(false);
            pool.Enqueue(obj);
        }
    }

    public GameObject Get(Vector3 pos, Quaternion rot)
    {
        var obj = pool.Count > 0 ? pool.Dequeue() : Instantiate(prefab);
        obj.transform.SetPositionAndRotation(pos, rot);
        obj.SetActive(true);
        return obj;
    }

    public void Return(GameObject obj) { obj.SetActive(false); pool.Enqueue(obj);
}
}
```

NOTA — UNITYENGINE.POOL
Unity 2021+ incluye UnityEngine.Pool.ObjectPool<T> integrado, con callbacks OnGet, OnRelease y OnDestroyPoolObject. Se recomienda esta implementación oficial en proyectos nuevos en lugar de implementaciones manuales.

2.3 Sistemas de inventario y economía: estructuras de datos, persistencia con JSON/PlayerPrefs y serialización

Un sistema de inventario gestiona los ítems que el jugador puede recolectar, usar, combinar o comerciar. La economía del juego define cómo se generan y consumen los recursos, afectando directamente el balance y la duración de la partida.

Estructura de datos del inventario

La elección de la estructura de datos afecta el rendimiento y la flexibilidad del sistema de inventario:

- **List<Item>:** Inventario simple y ordenado. Permite duplicados. Búsqueda O(n). Ideal para inventarios pequeños.
- **Dictionary<string, Item>:** Acceso por ID en O(1). Ideal para inventarios grandes donde se busca por nombre o ID único.
- **Dictionary<Item, int>:** Para ítems apilables (stackable). El valor entero representa la cantidad acumulada.
- **ScriptableObject como ítem:** Cada ítem es un asset .asset con sus propiedades. El inventario guarda solo referencias y cantidades.

```
// Definición de ítem como ScriptableObject
[CreateAssetMenu(menuName = "Inventory/Item")]
public class ItemData : ScriptableObject
{
    public string  itemId;
    public string  itemName;
    public string  description;
    public Sprite  icon;
```

```
    public bool    isStackable;
    public int     maxStack = 99;
    public float   weight;
}

public class Inventory : MonoBehaviour
{
    private Dictionary<ItemData, int> items = new();
    public event Action OnInventoryChanged;

    public bool AddItem(ItemData item, int qty = 1)
    {
        if (items.ContainsKey(item))
        {
            if (!item.isStackable) return false;
            items[item] = Mathf.Min(items[item] + qty, item.maxStack);
        }
        else items[item] = qty;
        OnInventoryChanged?.Invoke();
        return true;
    }

    public bool RemoveItem(ItemData item, int qty = 1)
    {
        if (!items.ContainsKey(item) || items[item] < qty) return false;
        items[item] -= qty;
        if (items[item] == 0) items.Remove(item);
        OnInventoryChanged?.Invoke();
        return true;
    }
}
```

Persistencia con PlayerPrefs

PlayerPrefs es el sistema más simple de Unity para guardar datos entre sesiones. Almacena pares clave-valor en el registro del sistema (Windows) o archivos de preferencias (macOS/Linux). Solo soporta int, float y string.

```
// Guardar
PlayerPrefs.SetInt("HighScore",    highScore);
PlayerPrefs.SetFloat("MusicVol",   0.8f);
PlayerPrefs.SetString("LastScene", "Level01");
PlayerPrefs.Save();  // forzar escritura inmediata a disco

// Leer con valor por defecto
int    score  = PlayerPrefs.GetInt("HighScore",    0);
float  vol    = PlayerPrefs.GetFloat("MusicVol",   1.0f);
string scene  = PlayerPrefs.GetString("LastScene", "MainMenu");

// Eliminar
PlayerPrefs.DeleteKey("HighScore");
PlayerPrefs.DeleteAll();  // borrar todo (usar con precaución)
```

IMPORTANTE

PlayerPrefs no es seguro: los datos se almacenan sin encriptación y pueden ser modificados por el usuario. No lo uses para datos sensibles (moneda premium, logros online) ni para estructuras complejas. Para guardar inventarios completos o el estado del mundo, usa serialización JSON.

Serialización con JSON

JSON permite serializar estructuras de datos complejas como texto y guardarlas en archivos. Unity incluye JsonUtility para serialización básica. Para casos avanzados (diccionarios, polimorfismo), se usa Newtonsoft.Json (Json.NET), disponible en Package Manager.

```
using System.IO;
using UnityEngine;

[System.Serializable]
public class SaveData
{
    public int          score;
    public int          lives;
    public float        playTime;
    public List<string> collectedItemIds;  // IDs de ítems recogidos
    public Vector3      playerPosition;
}

public static class SaveSystem
{
    static string Path => Application.persistentDataPath + "/save.json";

    public static void Save(SaveData data)
    {
        string json = JsonUtility.ToJson(data, prettyPrint: true);
        File.WriteAllText(Path, json);
    }

    public static SaveData Load()
    {
        if (!File.Exists(Path)) return new SaveData();
        return JsonUtility.FromJson<SaveData>(File.ReadAllText(Path));
    }

    public static void Delete()
    {
        if (File.Exists(Path)) File.Delete(Path);
    }
}
```

NOTA — APPLICATION.PERSISTENTDATAPATH

Esta ruta persiste entre actualizaciones de la app y no requiere permisos especiales. En Windows: %APPDATA%/CompanyName/ProductName. En macOS: ~/Library/Application Support/CompanyName/ProductName. En Android: /data/data/packagename/files.

2.4 Interfaces de usuario (UI Toolkit / uGUI): menús, HUD, barras de vida, diálogos y localización básica

Unity ofrece dos sistemas de UI: uGUI (sistema basado en GameObjects, disponible desde Unity 4.6) y UI Toolkit (sistema moderno basado en UXML/USS, similar a HTML/CSS). Para juegos 2D y 3D en producción se recomienda uGUI por su madurez; UI Toolkit es preferible para herramientas de editor y nuevos proyectos orientados a la web.

uGUI — Componentes fundamentales

Componente	Función	Uso típico

Canvas	Contenedor raíz de toda la UI	Un Canvas por pantalla (Screen/World Space)
CanvasScaler	Adapta la UI a distintas resoluciones	Scale with Screen Size, referencia 1920×1080
Image	Muestra sprites o texturas	Fondos, iconos de ítems, barras de vida
TextMeshPro	Texto de alta calidad con SDF rendering	Diálogos, HUD, botones, puntuación
Button	Elemento interactivo con evento onClick	Menús, acciones de inventario y habilidades
Slider	Control deslizante con valor float [0..1]	Barras de HP, maná, volumen, carga
ScrollRect	Área desplazable para contenido largo	Inventario, logs de misión, leaderboards
LayoutGroup	Organiza hijos automáticamente (H/V/Grid)	Slots de inventario, lista de opciones

HUD — Heads-Up Display

El HUD es la capa de UI que muestra información del estado del juego de forma persistente mientras el jugador juega. Debe ser legible, no intrusivo y actualizarse en tiempo real mediante eventos para no depender de polling en Update().

```
public class HUDManager : MonoBehaviour
{
    [SerializeField] private Slider   healthBar;
    [SerializeField] private TMP_Text scoreText;
    [SerializeField] private Image    healthFill;

    void OnEnable()
    {
        HealthSystem.OnHealthChanged += UpdateHealthBar;
        GameManager.OnScoreChanged   += UpdateScore;
    }
    void OnDisable()
    {
        HealthSystem.OnHealthChanged -= UpdateHealthBar;
        GameManager.OnScoreChanged   -= UpdateScore;
    }

    void UpdateHealthBar(float current, float max)
    {
        healthBar.value  = current / max;
        healthFill.color = current / max < 0.25f ? Color.red : Color.green;
    }

    void UpdateScore(int score) => scoreText.text = $"Score: {score:N0}";
}
```

Sistema de diálogos con efecto typewriter

Un sistema de diálogos muestra conversaciones de forma progresiva. El patrón más común usa una cola de líneas que se muestran con efecto de máquina de escribir, revelando los caracteres uno a uno.

```
public class DialogueSystem : MonoBehaviour
```

```
{
    [SerializeField] private GameObject dialoguePanel;
    [SerializeField] private TMP_Text   speakerText;
    [SerializeField] private TMP_Text   bodyText;
    [SerializeField] private float      charDelay = 0.03f;

    private Queue<DialogueLine> queue = new();
    private Coroutine typingCo;
    private bool isTyping;

    public void StartDialogue(DialogueData data)
    {
        queue.Clear();
        foreach (var line in data.lines) queue.Enqueue(line);
        dialoguePanel.SetActive(true);
        ShowNext();
    }

    public void OnContinue()
    {
        if (isTyping) { StopCoroutine(typingCo); isTyping = false; return; }
        if (queue.Count > 0) ShowNext();
        else dialoguePanel.SetActive(false);
    }

    void ShowNext()
    {
        var line = queue.Dequeue();
        speakerText.text = line.speaker;
        typingCo = StartCoroutine(TypeText(line.text));
    }

    IEnumerator TypeText(string text)
    {
        isTyping = true;
        bodyText.text = "";
        foreach (char c in text)
        {
            bodyText.text += c;
            yield return new WaitForSeconds(charDelay);
        }
        isTyping = false;
    }
}
```

Localización básica con el paquete Localization

El paquete oficial Unity Localization (com.unity.localization) gestiona tablas de cadenas, traducciones y localización de assets (sprites, audio) para múltiples idiomas.

- **String Table:** Tabla de pares clave-valor por idioma. Clave 'ui_start_button' → 'Iniciar' (es) / 'Start' (en).
- **LocalizeStringEvent:** Componente que conecta un TextMeshPro a una entrada de la String Table y actualiza automáticamente al cambiar de Locale.
- **Locale:** Asset que representa un idioma (es-BO, en-US, pt-BR). LocalizationSettings gestiona el Locale activo en tiempo de ejecución.

```
using UnityEngine.Localization.Settings;
using System.Linq;

// Cambiar idioma en tiempo de ejecución
```

```
async void SetLanguage(string code)  // "es", "en", "pt"
{
    await LocalizationSettings.InitializationOperation.Task;
    var locales  = LocalizationSettings.AvailableLocales.Locales;
    var selected = locales.FirstOrDefault(l => l.Identifier.Code == code);
    if (selected != null) LocalizationSettings.SelectedLocale = selected;
}
```

2.5 Gestión de escenas y flujo de juego: carga aditiva, pantallas de carga, transiciones y persistencia entre escenas

La gestión de escenas controla cómo el juego transita entre sus diferentes estados: menú principal, niveles, pantallas de carga, game over y créditos. Una gestión deficiente causa tiempos de carga notorios, pérdida de datos entre escenas y referencias rotas.

SceneManager — carga básica y asíncrona

```
using UnityEngine.SceneManagement;

// Carga síncrona (bloquea — solo para escenas muy pequeñas)
SceneManager.LoadScene("Level01");
SceneManager.LoadScene(1);  // por índice configurado en Build Settings

// Carga asíncrona con barra de progreso
IEnumerator LoadAsync(string sceneName)
{
    AsyncOperation op = SceneManager.LoadSceneAsync(sceneName);
    op.allowSceneActivation = false;

    while (!op.isDone)
    {
        float progress = Mathf.Clamp01(op.progress / 0.9f);
        loadingBar.value = progress;

        if (op.progress >= 0.9f)  // escena lista, activar con pequeña pausa
visual
        {
            yield return new WaitForSeconds(0.5f);
            op.allowSceneActivation = true;
        }
        yield return null;
    }
}
```

Carga aditiva (Additive Loading)

La carga aditiva permite tener múltiples escenas activas simultáneamente sin descargar la actual. Es fundamental para streaming de mundos grandes, UI persistente entre niveles y arquitecturas de escenas modulares.

```
// Cargar menú de pausa sin descargar la escena del juego
SceneManager.LoadScene("PauseMenu", LoadSceneMode.Additive);

// Descargar cuando se cierra el menú
SceneManager.UnloadSceneAsync("PauseMenu");

// Arquitectura recomendada para proyectos medianos:
```

```
// Persistent  — GameManager, AudioManager, UIManager (DontDestroyOnLoad)
// MainMenu    — solo mientras estás en el menú
// Level01     — geometría, enemigos y lógica del nivel
// HUD         — cargada de forma aditiva junto al nivel, persiste entre niveles
```

Métodos de persistencia entre escenas

Método	Cuándo usarlo	Limitaciones
DontDestroyOnLoad	Managers globales (GameManager, AudioManager)	Cuidar duplicados al recargar escena
PlayerPrefs	Configuración, high scores, progresión simple	Solo int/float/string, sin encriptación
JSON en disco	Estado completo, inventario, posición del mundo	Lento si se llama en Update()
ScriptableObject	Datos de configuración compartidos en editor	No persiste entre ejecuciones standalone
Static variables	Datos simples de transferencia entre escenas	Dificulta testing, no serializable

Transiciones con fade in/out

```
public class SceneTransitioner : MonoBehaviour
{
    public static SceneTransitioner Instance { get; private set; }
    [SerializeField] private Animator fadeAnim;
    static readonly int HashOut = Animator.StringToHash("FadeOut");
    static readonly int HashIn  = Animator.StringToHash("FadeIn");

    void Awake()
    {
        if (Instance != null) { Destroy(gameObject); return; }
        Instance = this;
        DontDestroyOnLoad(gameObject);
    }

    public void TransitionTo(string sceneName) =>
        StartCoroutine(DoTransition(sceneName));

    IEnumerator DoTransition(string sceneName)
    {
        fadeAnim.SetTrigger(HashOut);
        yield return new WaitForSeconds(0.5f);
        var op = SceneManager.LoadSceneAsync(sceneName);
        yield return new WaitUntil(() => op.isDone);
        fadeAnim.SetTrigger(HashIn);
    }
}

// Uso: SceneTransitioner.Instance.TransitionTo("Level02");
```

2.6 Prototipo 1: Juego 2D Top-Down — integración de mecánicas, UI y gestión de escenas

El Prototipo 1 es el proyecto integrador de la Unidad 2. Consiste en un juego 2D de vista superior (top-down) que puede adoptar la forma de un RPG de exploración o un Shooter con oleadas de enemigos. El objetivo es demostrar que todos los sistemas aprendidos en la unidad funcionan de forma cohesionada en un producto jugable.

Especificación del prototipo

Sistema	Requisito mínimo	Tecnología
Movimiento del jugador	8 direcciones con colisiones físicas	Rigidbody2D + Input System
IA de enemigos	Al menos 2 estados: Patrol y Chase	State Machine manual
Sistema de combate	Disparo de proyectiles con daño y destrucción	Object Pool + Collider2D
Inventario/recursos	Recolección de ítems y conteo de recursos	Dictionary<ItemData, int>
HUD	HP, puntuación y recurso principal visibles	uGUI + TMP + Slider
Menú principal	Pantalla de inicio con botones Play y Quit	Canvas + Button + SceneManager
Game Over / Victoria	Pantalla con puntuación y opción de reinicio	Condición de victoria + SceneManager
Guardado básico	Guardar y cargar puntuación máxima	PlayerPrefs o JSON
Transiciones	Fade entre escenas	Animator + SceneTransitioner

Arquitectura de escenas del prototipo

- **Persistent:** GameManager (Singleton + DontDestroyOnLoad), AudioManager, SceneTransitioner. Se carga primero y nunca se destruye.
- **MainMenu:** Canvas con título, botones Play, Options y Quit. Fondo animado simple.
- **Level01:** Tilemap del nivel, spawn de jugador y enemigos, lógica de oleadas, trigger de victoria.
- **GameOver:** Puntuación final, récord guardado, botones Retry y Main Menu.

Estructura de scripts recomendada

```
Assets/_Project/Scripts/
├── Core/
│   ├── GameManager.cs          // Singleton: estado global, score, lives
│   ├── AudioManager.cs         // Singleton: SFX y música
│   └── SceneTransitioner.cs    // Singleton: fade y carga de escenas
├── Player/
│   ├── PlayerController.cs     // Movimiento 8 dirs con Input System
│   ├── PlayerHealth.cs         // HP, daño, muerte y eventos
│   ├── PlayerShooter.cs        // Disparo con Object Pool
│   └── PlayerInventory.cs      // Recolección y uso de ítems
├── Enemies/
│   ├── EnemyAI.cs              // State Machine: Patrol / Chase / Attack
│   ├── EnemyHealth.cs          // HP, muerte y drop de ítems
│   └── EnemySpawner.cs         // Oleadas con delay y spawn points
├── Weapons/
│   ├── Projectile.cs           // Movimiento, daño y retorno al pool
│   └── ObjectPool.cs           // Pool genérico reutilizable
```

```
├── UI/
│   ├── HUDManager.cs           // HP bar, score, resource counter
│   ├── MainMenuUI.cs           // Botones Play/Options/Quit
│   └── GameOverUI.cs           // Score final, high score, Retry
└── Data/
    ├── ItemData.cs             // ScriptableObject: definición de ítems
    └── SaveSystem.cs           // JSON save/load
```

Flujo de juego del prototipo

- **1. Arranque:** Se carga Persistent → GameManager inicializa estado → MainMenu con fade in.
- **2. Play:** El jugador presiona Play → SceneTransitioner fade out → carga Level01 → fade in.
- **3. Gameplay:** Jugador se mueve, dispara y recoge ítems. EnemySpawner genera oleadas. HUD actualiza via eventos.
- **4. Victoria:** Todos los enemigos derrotados → GameManager.TriggerVictory() → carga GameOver con flag de victoria.
- **5. Derrota:** HP = 0 → PlayerHealth dispara OnPlayerDeath → GameManager.TriggerGameOver() → carga GameOver.
- **6. Game Over:** GameOverUI muestra puntuación actual vs. máxima guardada. Retry recarga Level01. Main Menu vuelve al inicio.

> **CONSEJO DE PROTOTIPADO**
> Al desarrollar el prototipo, prioriza primero que el gameplay se sienta divertido antes de pulir el arte. Usa sprites de placeholder (cuadrados de colores) para el jugador y los enemigos. Una mecánica de disparo y movimiento que se sienta bien es más valiosa en esta etapa que sprites detallados. El arte puede mejorarse en iteraciones posteriores sin afectar la arquitectura del código.